

```
import numpy as np
import matplotlib.pyplot as plt
```

```
#Create Dataset (Student Marks)
data = np.array([
    [85, 78, 92, 88, 76],
    [79, 85, 80, 90, 88],
    [92, 89, 94, 91, 87],
    [70, 75, 78, 72, 74],
    [88, 84, 86, 85, 90]
])

print("Dataset:\n", data)
```

```
Dataset:
[[85 78 92 88 76]
 [79 85 80 90 88]
 [92 89 94 91 87]
 [70 75 78 72 74]
 [88 84 86 85 90]]
```

```
#Matrix Operations
print("Transpose:\n", data.T)

print("\nMatrix Multiplication:\n", np.dot(data, data.T))

print("\nDeterminant (3x3):\n", np.linalg.det(data[:3, :3]))

print("\nInverse (3x3):\n", np.linalg.inv(data[:3, :3]))
```

```
Transpose:
[[85 79 92 70 88]
 [78 85 89 75 84]
 [92 80 94 78 86]
 [88 90 91 72 85]
 [76 88 87 74 90]]

Matrix Multiplication:
[[35293 35313 38030 30936 36264]
 [35313 35710 38199 31137 36542]
 [38030 38199 41071 33437 39221]
 [30936 31137 33437 27269 31948]
 [36264 36542 39221 31948 37521]]

Determinant (3x3):
-3786.00000000013

Inverse (3x3):
[[-0.22979398 -0.22609614  0.41732699]
 [ 0.01743265  0.1251981  -0.12361331]
 [ 0.20839937  0.10274696 -0.28077126]]
```

```
#Horizontal & Vertical Stacking
extra = np.array([[80, 82, 84, 86, 88]])

print("Vertical Stack:\n", np.vstack((data, extra)))

print("\nHorizontal Stack:\n", np.hstack((data, data)))
```

```
Vertical Stack:
[[85 78 92 88 76]
 [79 85 80 90 88]
 [92 89 94 91 87]
 [70 75 78 72 74]
 [88 84 86 85 90]
 [80 82 84 86 88]]

Horizontal Stack:
[[85 78 92 88 76 85 78 92 88 76]
 [79 85 80 90 88 79 85 80 90 88]
 [92 89 94 91 87 92 89 94 91 87]
 [70 75 78 72 74 70 75 78 72 74]
 [88 84 86 85 90 88 84 86 85 90]]
```

```
#Custom Sequence Generation
print("Arange:", np.arange(0, 20, 2))

print("Linspace:", np.linspace(0, 1, 5))

print("Random Matrix:\n", np.random.randint(50, 100, (3,3)))
```

```
Arange: [ 0  2  4  6  8 10 12 14 16 18]
Linspace: [0.   0.25 0.5  0.75 1. ]
Random Matrix:
[[83 66 99]
 [75 50 76]
 [59 85 69]]
```

```
#Arithmetic Operations
print("Addition:\n", data + 5)
print("\nSubtraction:\n", data - 5)
print("\nMultiplication:\n", data * 2)
print("\nDivision:\n", data / 2)
```

```
Addition:
[[90 83 97 93 81]
 [84 90 85 95 93]
 [97 94 99 96 92]
 [75 80 83 77 79]
 [93 89 91 90 95]]
```

```
Subtraction:
[[80 73 87 83 71]
 [74 80 75 85 83]
 [87 84 89 86 82]
 [65 70 73 67 69]
 [83 79 81 80 85]]
```

```
Multiplication:
[[170 156 184 176 152]
 [158 170 160 180 176]
 [184 178 188 182 174]
 [140 150 156 144 148]
 [176 168 172 170 180]]
```

```
Division:
[[42.5 39.  46.  44.  38. ]
 [39.5 42.5 40.  45.  44. ]
 [46.  44.5 47.  45.5 43.5]
 [35.  37.5 39.  36.  37. ]
 [44.  42.  43.  42.5 45. ]]
```

```
#Statistical Operations
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))
```

```
Mean: 83.84
Median: 85.0
Standard Deviation: 6.73902070036886
Minimum: 70
Maximum: 94
```

```
#Mathematical Operations
print("Square:\n", np.square(data))

print("\nSquare Root:\n", np.sqrt(data))

print("\nLog Values:\n", np.log(data))
```

```
Square:
[[7225 6084 8464 7744 5776]
 [6241 7225 6400 8100 7744]
 [8464 7921 8836 8281 7569]
 [4900 5625 6084 5184 5476]
 [7744 7056 7396 7225 8100]]
```

```
Square Root:
[[9.21954446 8.83176087 9.59166305 9.38083152 8.71779789]
 [8.88819442 9.21954446 8.94427191 9.48683298 9.38083152]
 [9.59166305 9.43398113 9.69535971 9.53939201 9.32737905]
 [8.36660027 8.66025404 8.83176087 8.48528137 8.60232527]
 [9.38083152 9.16515139 9.2736185  9.21954446 9.48683298]]
```

```
Log Values:
[[4.44265126 4.35670883 4.52178858 4.47733681 4.33073334]
 [4.36944785 4.44265126 4.38202663 4.49980967 4.47733681]
 [4.52178858 4.48863637 4.54329478 4.51085951 4.46590812]
 [4.24849524 4.31748811 4.35670883 4.27666612 4.30406509]
 [4.47733681 4.4308168  4.4543473  4.44265126 4.49980967]]
```

```
#Bitwise Operations
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
```

```
print("AND:", np.bitwise_and(a, b))
print("OR:", np.bitwise_or(a, b))
print("XOR:", np.bitwise_xor(a, b))
```

```
AND: [0 0 2]
OR: [5 7 7]
XOR: [5 7 5]
```

#Copying vs Viewing Arrays

```
copy_arr = data.copy()
view_arr = data.view()
```

```
copy_arr[0][0] = 999
```

```
print("Original Array:\n", data)
print("\nCopied Array:\n", copy_arr)
print("\nViewed Array:\n", view_arr)
```

```
Original Array:
[[85 78 92 88 76]
 [79 85 80 90 88]
 [92 89 94 91 87]
 [70 75 78 72 74]
 [88 84 86 85 90]]
```

```
Copied Array:
[[999 78 92 88 76]
 [ 79 85 80 90 88]
 [ 92 89 94 91 87]
 [ 70 75 78 72 74]
 [ 88 84 86 85 90]]
```

```
Viewed Array:
[[85 78 92 88 76]
 [79 85 80 90 88]
 [92 89 94 91 87]
 [70 75 78 72 74]
 [88 84 86 85 90]]
```

#Searching, Sorting, Counting

```
print("Sorted Data:\n", np.sort(data))
```

```
print("\nIndices where values > 85:\n", np.where(data > 85))
```

```
print("\nCount of values > 80:", np.sum(data > 80))
```

```
Sorted Data:
[[76 78 85 88 92]
 [79 80 85 88 90]
 [87 89 91 92 94]
 [70 72 74 75 78]
 [84 85 86 88 90]]
```

```
Indices where values > 85:
(array([0, 0, 1, 1, 2, 2, 2, 2, 2, 4, 4, 4]), array([2, 3, 3, 4, 0, 1, 2, 3, 4, 0, 2, 4]))
```

```
Count of values > 80: 16
```

#Broadcasting

```
print("Broadcasting (Row Addition):\n", data + np.array([1,2,3,4,5]))
```

```
Broadcasting (Row Addition):
[[86 80 95 92 81]
 [80 87 83 94 93]
 [93 91 97 95 92]
 [71 77 81 76 79]
 [89 86 89 89 95]]
```

#Data Stacking

```
stacked = np.stack((data, data))
```

```
print("Stacked Array:\n", stacked)
```

```
Stacked Array:
[[[85 78 92 88 76]
 [79 85 80 90 88]
 [92 89 94 91 87]
 [70 75 78 72 74]
 [88 84 86 85 90]]
```

```
[[85 78 92 88 76]
 [79 85 80 90 88]]
```

```
[92 89 94 91 87]  
[70 75 78 72 74]  
[88 84 86 85 90]]]
```